

VR_week7Demo

February 24, 2026

1 For PF Section

2 Data Visualization Week-7

- BTech Computer Science Stream , Februry 2026
- Week 7 - Plotting and Visualization using Matplotlib & Seaborn Exercise
- Name: Dr. Vidya Rao
- Reg Number: 12345678
- Date: 26/02/2026

2.1 Learning Outcomes

By the end of this session, you will be able to:

- Load and inspect datasets
- Perform aggregation using groupby()
- Extract Top-N records
- Create bar, scatter, histogram, box and pie charts
- Create derived columns
- Add annotations and storytelling insights

2.2 Workflow

- Load → Inspect → Aggregate → Visualize → Interpret

2.3 Step 1: Understand Dataset Structure

Before plotting: - How many rows? - What are column types? - Which are categorical? - Which are numeric?

2.4 Setting up the enviroment

```
[32]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Optional: make plots appear larger by default
plt.rcParams["figure.figsize"] = (8, 5)
```

```
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 11

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 1200)

print("Setup complete.")
```

Setup complete.

2.5 Upload the dataset

```
[33]: df=pd.read_csv('mtcars.csv')
pd.set_option('display.max_columns', 100) #display all columns
pd.set_option('display.max_rows',None)   #display all rows
```

```
[34]: df
```

```
[34]:
```

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear
carb											
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4
4											
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4
4											
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4
1											
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3
1											
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3
2											
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3
1											
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3
4											
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4
2											
8	Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4
2											
9	Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4
4											
10	Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4
4											
11	Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3
3											
12	Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3
3											
13	Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3

3												
14	Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	
4												
15	Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	
4												
16	Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	
4												
17	Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	
1												
18	Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	
2												
19	Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	
1												
20	Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	
1												
21	Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	
2												
22	AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	
2												
23	Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	
4												
24	Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	
2												
25	Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	
1												
26	Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	
2												
27	Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	
2												
28	Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	
4												
29	Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	
6												
30	Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	
8												
31	Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	
2												

```
[35]: print(df.shape)
      print(df.info())
      print(df.describe())
```

```
(32, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 12 columns):
 #   Column  Non-Null Count  Dtype
---  -

```

```

0  model    32 non-null    object
1  mpg      32 non-null    float64
2  cyl      32 non-null    int64
3  disp     32 non-null    float64
4  hp       32 non-null    int64
5  drat     32 non-null    float64
6  wt       32 non-null    float64
7  qsec     32 non-null    float64
8  vs       32 non-null    int64
9  am       32 non-null    int64
10 gear     32 non-null    int64
11 carb     32 non-null    int64
dtypes: float64(5), int64(6), object(1)
memory usage: 3.1+ KB
None

```

	mpg	cyl	disp	hp	drat	wt
qsec	vs	am	gear	carb		
count	32.000000	32.000000	32.000000	32.000000	32.000000	32.000000
mean	20.090625	6.187500	230.721875	146.687500	3.596563	3.217250
std	6.026948	1.785922	123.938694	68.562868	0.534679	0.978457
min	10.400000	4.000000	71.100000	52.000000	2.760000	1.513000
25%	15.425000	4.000000	120.825000	96.500000	3.080000	2.581250
50%	19.200000	6.000000	196.300000	123.000000	3.695000	3.325000
75%	22.800000	8.000000	326.000000	180.000000	3.920000	3.610000
max	33.900000	8.000000	472.000000	335.000000	4.930000	5.424000

2.6 Counting & Unique Analysis

```
[36]: df.nunique()
```

```

[36]: model    32
      mpg      25
      cyl      3
      disp     27
      hp       22
      drat     22
      wt       29
      qsec     30
      vs       2

```

```
am          2
gear        3
carb        6
dtype: int64
```

```
[37]: print("Total Cars:", len(df))
      print("Unique Cylinder Types:", df["cyl"].nunique())
      print("Transmission Types:", df["am"].nunique())
```

```
Total Cars: 32
Unique Cylinder Types: 3
Transmission Types: 2
```

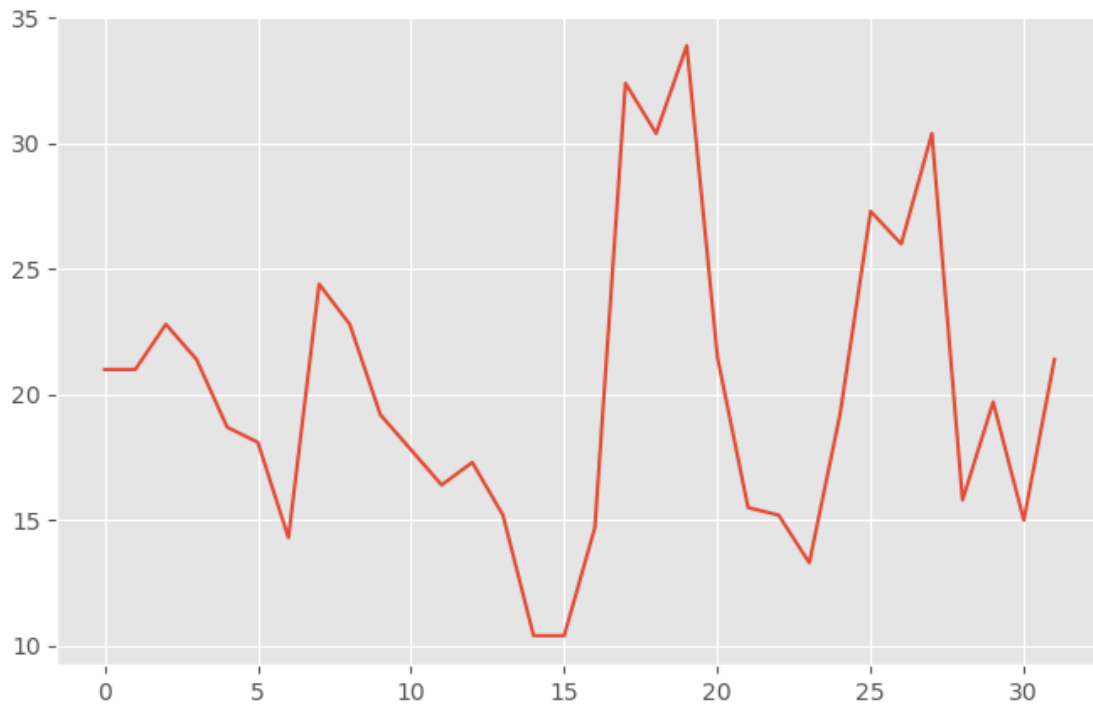
2.7 Groupby Analysis

```
[38]: cyl_counts = df.groupby("cyl")["mpg"].count()
      cyl_counts
```

```
[38]: cyl
      4      11
      6       7
      8      14
      Name: mpg, dtype: int64
```

```
[39]: # Extract the 'mpg' column from the dataframe & Plot the mpg values as a line
      ↳ plot
      mpg = df['mpg']
      mpg.plot()
      # The default plot will be a line plot where the x-axis represents the index of
      ↳ the data points
      # and the y-axis represents the 'mpg' values.
```

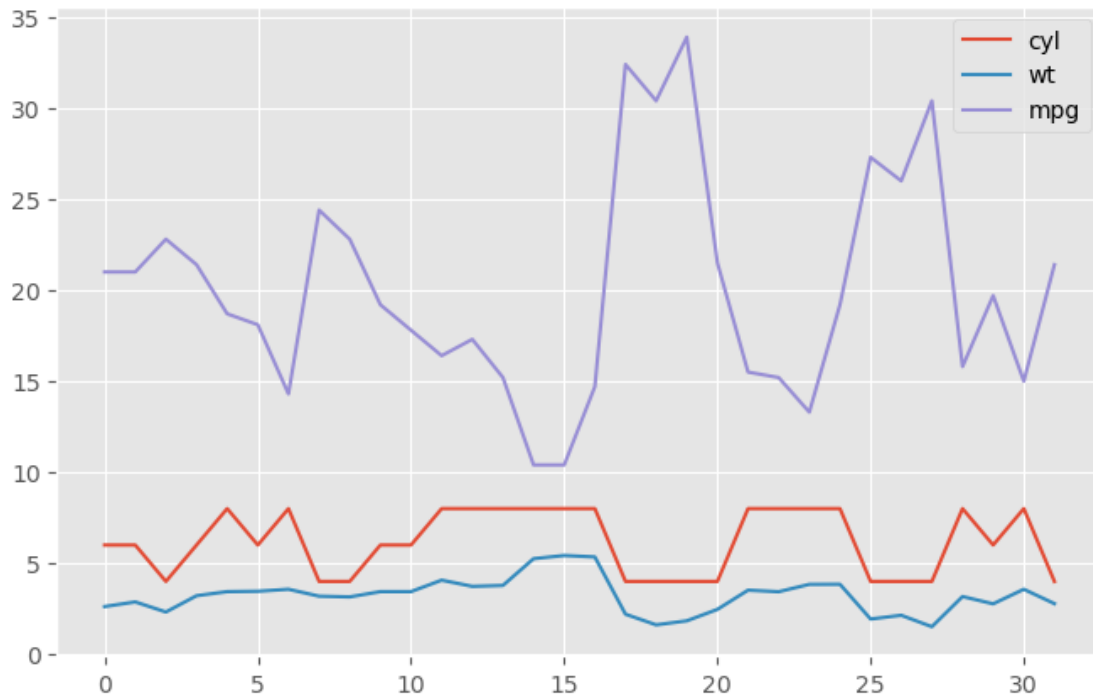
```
[39]: <Axes: >
```



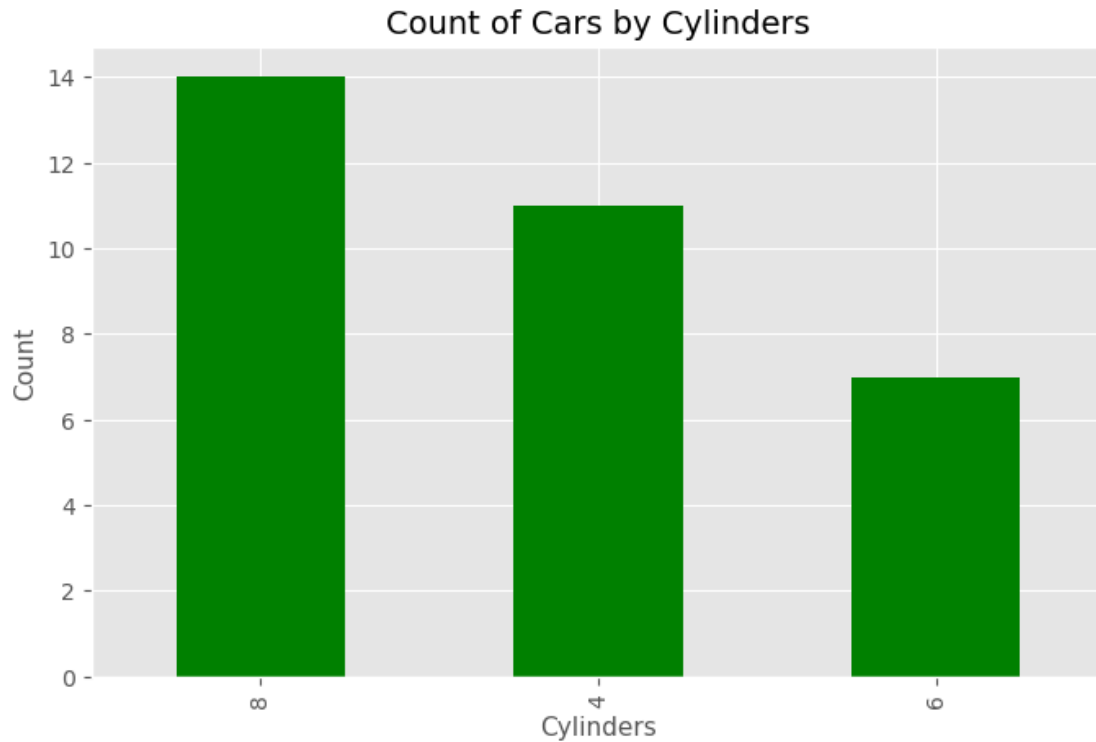
```
[40]: # Select only the 'cyl', 'wt', and 'mpg' columns from the dataframe & Plot them
      ↪ selected dataframe columns
df = df[['cyl', 'wt', 'mpg']]
df.plot()

# By default, Pandas will create a line plot for all numerical columns ('cyl',
  ↪ 'wt', 'mpg')
# - The x-axis represents the index of the dataframe.
# - The y-axis represents the values of 'cyl', 'wt', and 'mpg'.
# - Each column gets a separate line in the plot.
```

```
[40]: <Axes: >
```

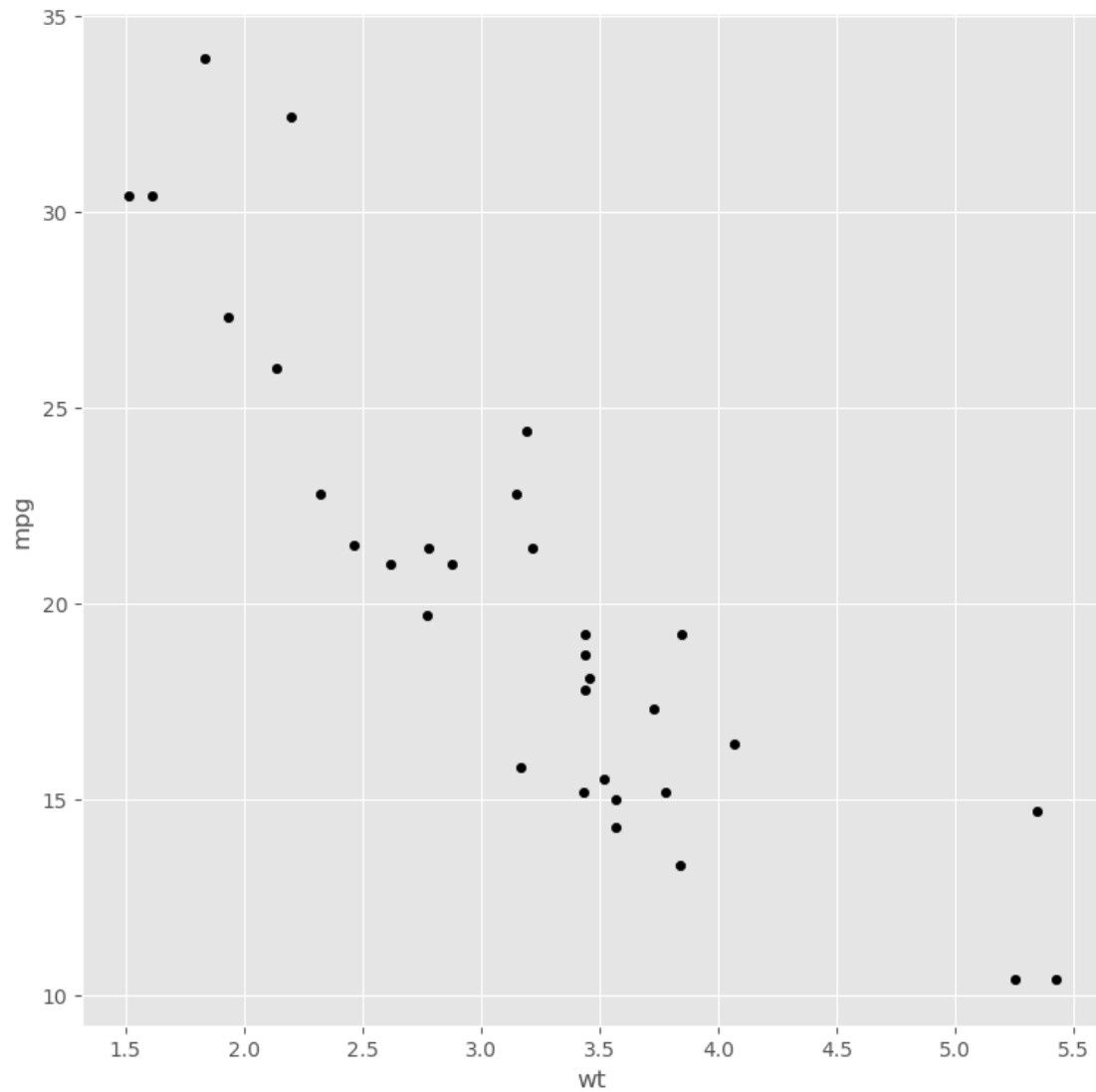


```
[41]: #Bar Plot- the count of cars by the number of cylinders
cyl_counts = df['cyl'].value_counts()
cyl_counts.plot(kind='bar', color='green')
plt.title('Count of Cars by Cylinders')
plt.xlabel('Cylinders')
plt.ylabel('Count')
plt.show()
```

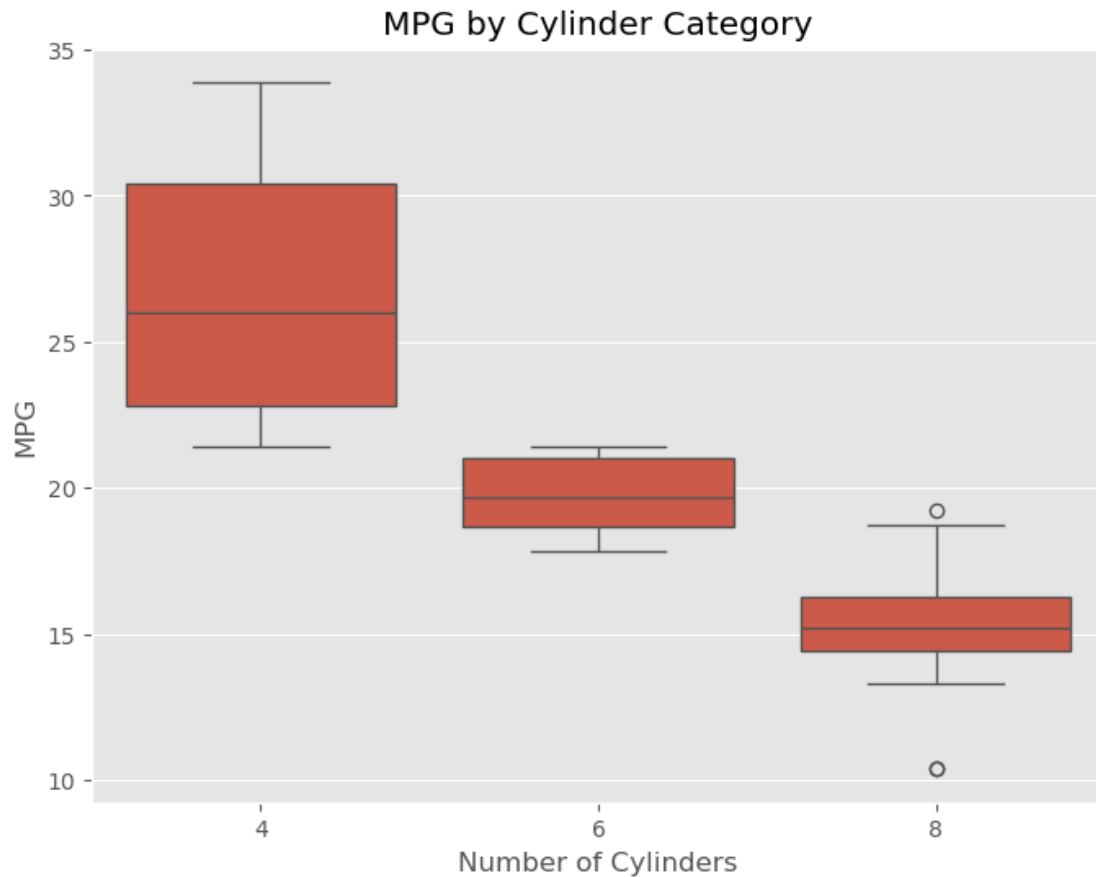


```
[42]: #use ggplot for better visualization. ggplot2 is based on the Grammar of
      ↪Graphics, which provides a structured approach to building plots.
      # This makes it easy to add, remove, or modify components (e.g., layers,
      ↪scales, themes) without breaking the code.
      plt.style.use('ggplot')
```

```
[43]: #Scatter plot of weight vs mpg
      df.plot(
          kind="scatter",
          x="wt",
          y="mpg",
          figsize=(9,9),
          color="black",
      );
```



```
[44]: #Box plot of MPG for each cylinder category
plt.figure(figsize=(8, 6))
sns.boxplot(x='cyl', y='mpg', data=df)
plt.title('MPG by Cylinder Category')
plt.xlabel('Number of Cylinders')
plt.ylabel('MPG')
plt.show()
```



```
[45]: # Histogram of MPG for all cars

x = df['mpg']

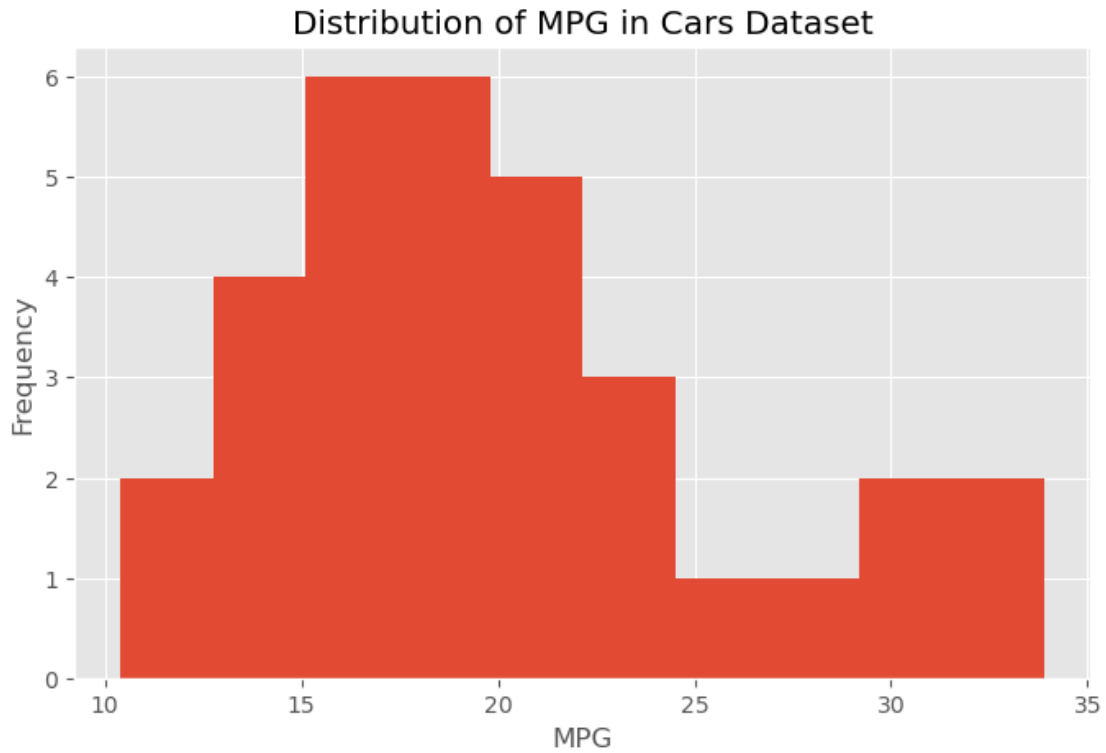
# Plotting the histogram
plt.hist(x)

# Adding title to the plot
plt.title("Distribution of MPG in Cars Dataset")

# Adding label on the y-axis (Frequency of each cylinder count)
plt.ylabel('Frequency')

# Adding label on the x-axis (Number of cylinders in cars)
plt.xlabel('MPG')

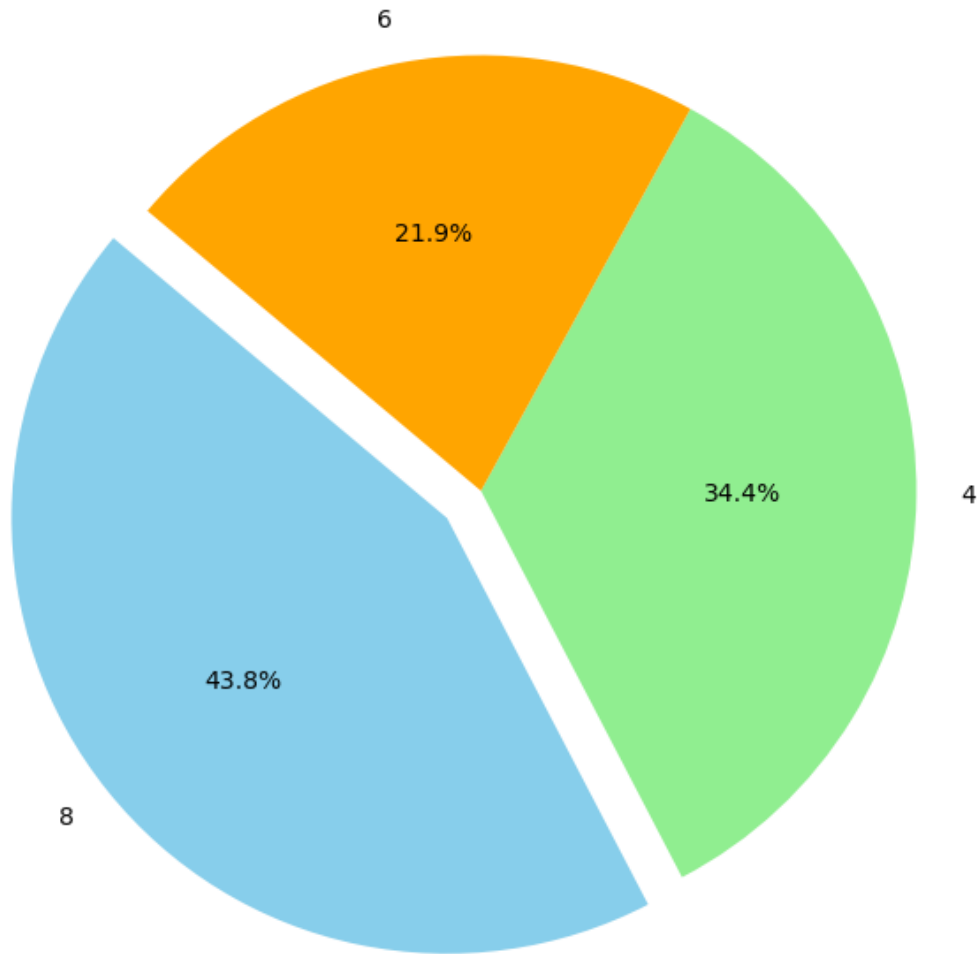
# Displaying the plot
plt.show()
```



```
[46]: # This pie chart shows the proportion of cars with 4, 6, or 8 cylinders.
# Group data by 'cyl' and calculate counts
cyl_counts = df['cyl'].value_counts()

# Create a pie chart
plt.figure(figsize=(8, 8))
plt.pie(
    cyl_counts,
    labels=cyl_counts.index,
    autopct='%1.1f%%', # Add percentage labels
    colors=['skyblue', 'lightgreen', 'orange'],
    startangle=140, # Start angle of the pie chart
    explode=[0.1, 0, 0] # Emphasize the first slice (4 cylinders)
)
plt.title('Proportion of Cars by Number of Cylinders')
plt.show()
```

Proportion of Cars by Number of Cylinders



```
[47]: # Pie Chart for Transmission Types

# Reload the original dataframe to include the 'am' column
df = pd.read_csv('mtcars.csv')

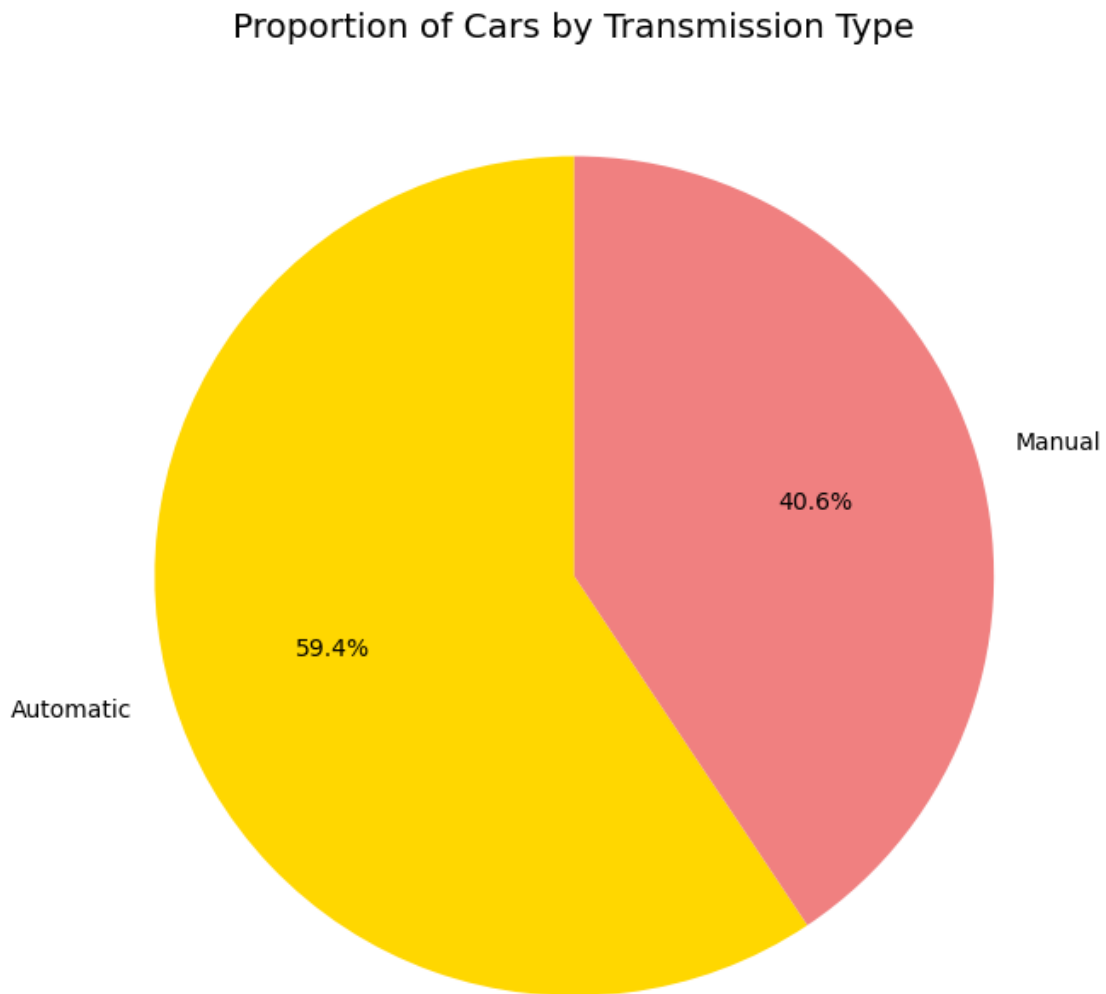
am_counts = df['am'].value_counts()

# Create a pie chart
plt.figure(figsize=(8, 8))
plt.pie(
```

```

am_counts,
labels=['Automatic', 'Manual'], # Custom labels
autopct='%1.1f%%',
colors=['gold', 'lightcoral'],
startangle=90
)
plt.title('Proportion of Cars by Transmission Type')
plt.show()

```



```

[48]: # Nested Pie Chart which needs Data preparation
outer_labels = df['cyl'].value_counts().index
outer_sizes = df['cyl'].value_counts().values

```

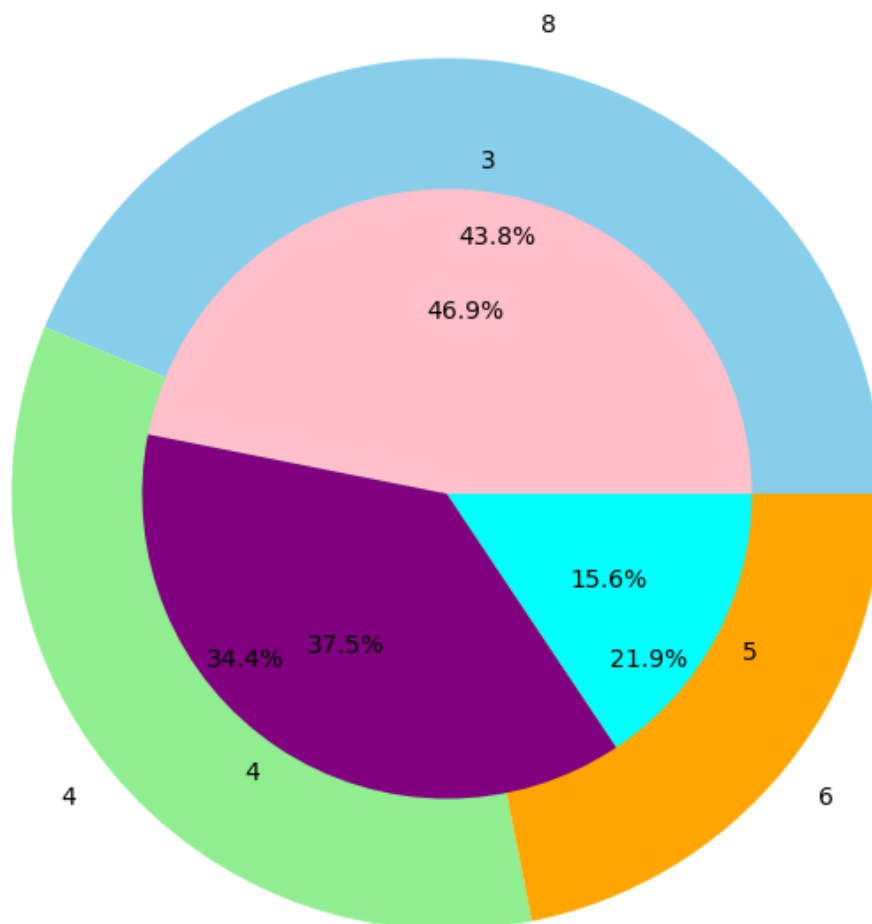
```

inner_labels = df['gear'].value_counts().index
inner_sizes = df['gear'].value_counts().values

# Create nested pie chart
fig, ax = plt.subplots(figsize=(8, 8))
ax.pie(outer_sizes, labels=outer_labels, radius=1, autopct='%1.1f%%',
      colors=['skyblue', 'lightgreen', 'orange'])
ax.pie(inner_sizes, labels=inner_labels, radius=0.7, autopct='%1.1f%%',
      colors=['pink', 'purple', 'cyan'])
ax.set(aspect="equal")
plt.title('Nested Pie Chart: Cylinders (Outer) and Gears (Inner)')
plt.show()

```

Nested Pie Chart: Cylinders (Outer) and Gears (Inner)



2.8 Top-N Analysis

```
[49]: #Sorting
top_hp = df.sort_values("hp", ascending=False).head(5)
top_hp[["model", "hp"]]
```

```
[49]:
```

	model	hp
30	Maserati Bora	335
28	Ford Pantera L	264
6	Duster 360	245
23	Camaro Z28	245
16	Chrysler Imperial	230

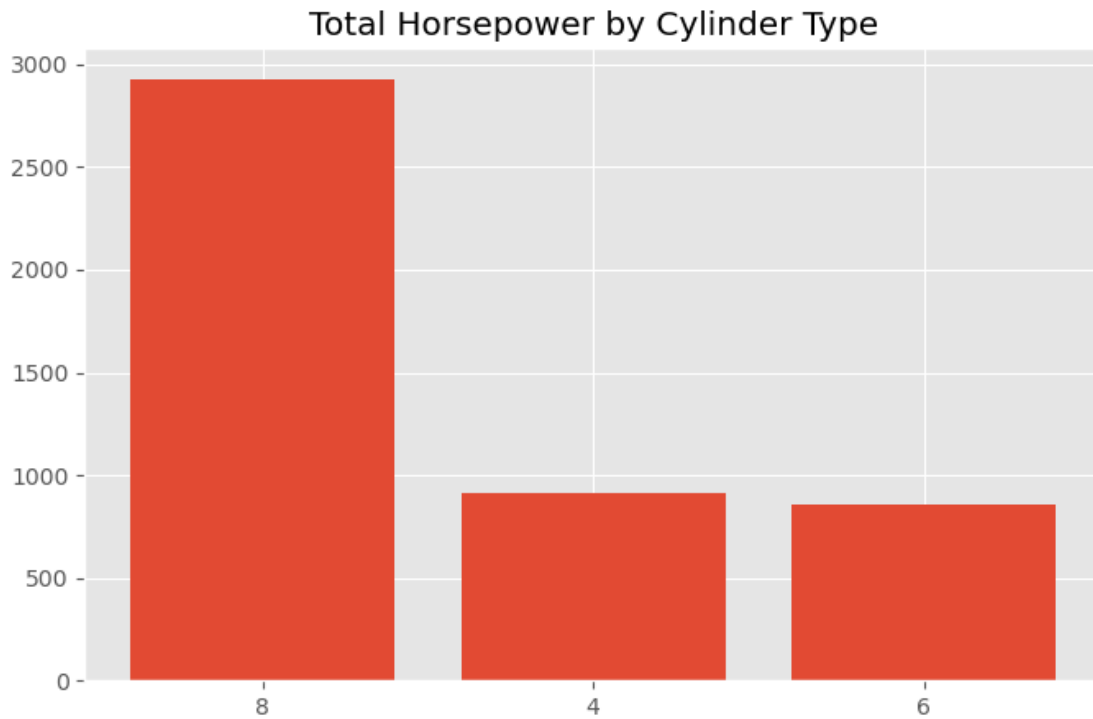
```
[50]: #Category Top-N

df["cyl"].value_counts().head(2)
```

```
[50]: cyl
      8    14
      4    11
      Name: count, dtype: int64
```

```
[51]: top_power = df.groupby("cyl")["hp"].sum().sort_values(ascending=False)

plt.bar(top_power.index.astype(str), top_power.values)
plt.title("Total Horsepower by Cylinder Type")
plt.show()
```



2.9 Creating Derived Columns

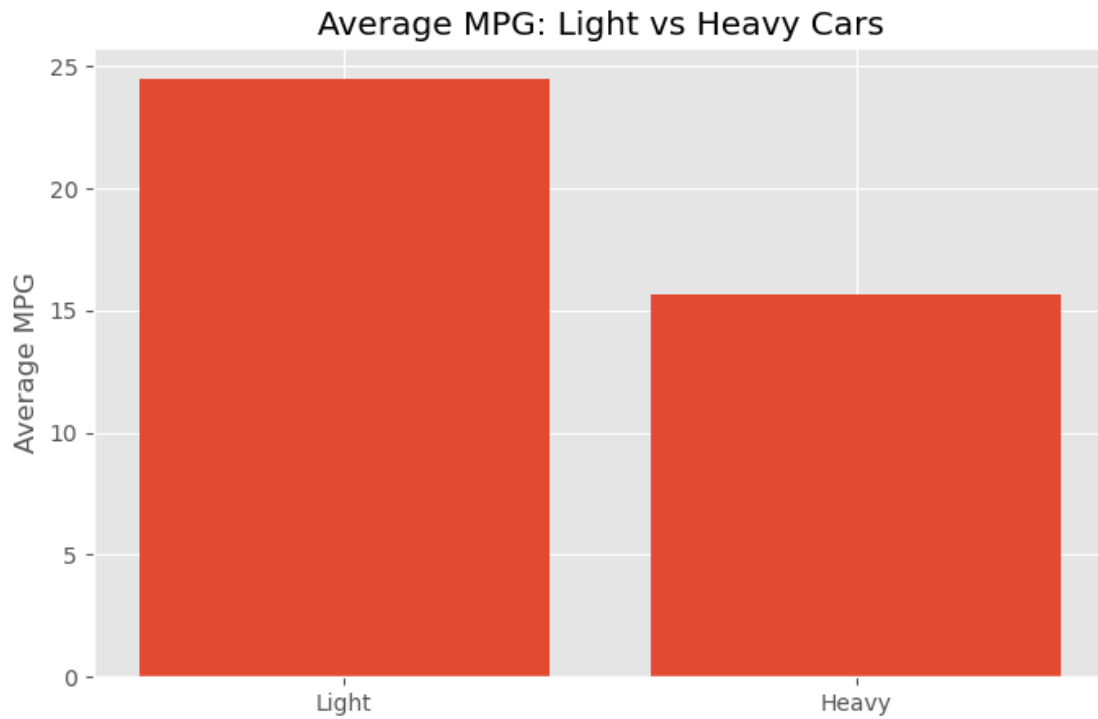
```
[52]: df["heavy_car"] = df["wt"] > df["wt"].mean()
      df["heavy_car"].value_counts()
```

```
[52]: heavy_car
      False    16
       True     16
      Name: count, dtype: int64
```

```
[53]: #Comparison Plot

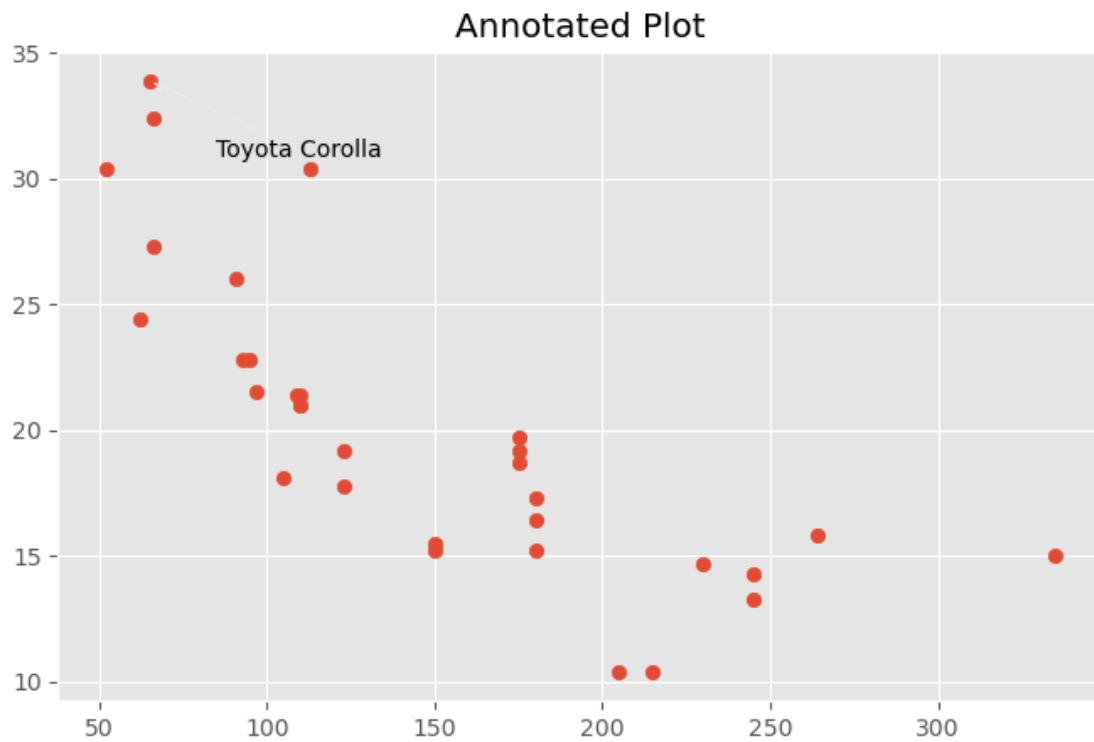
      grp = df.groupby("heavy_car")["mpg"].mean()

      plt.bar(["Light", "Heavy"], grp.values)
      plt.title("Average MPG: Light vs Heavy Cars")
      plt.ylabel("Average MPG")
      plt.show()
```



```
[54]: best = df.loc[df["mpg"].idxmax()]

plt.scatter(df["hp"], df["mpg"])
plt.annotate(best["model"],
             (best["hp"], best["mpg"]),
             xytext=(best["hp"]+20, best["mpg"]-3),
             arrowprops=dict(arrowstyle="->"))
plt.title("Annotated Plot")
plt.show()
```



```
[55]: plt.scatter(df["hp"], df["mpg"])
plt.title("Horsepower vs MPG")
plt.xlabel("HP")
plt.ylabel("MPG")
plt.grid(True)
plt.show()
```

